Programación I

TEMA 3. Estructuras de control repetitivas

Profesores

- Lourdes Borrajo (lborrajo@uvigo.gal)
- Iván Luis (ivan.luis@uvigo.es)

Universidade de Vigo

¿Qué son las estructuras de control?

Las **estructuras de control** permiten que un programa:

- Tome **decisiones**.
- Repita acciones (**bucles**).
- Ejecute instrucciones de forma **ordenada y lógica**.

> ⚠️ Las estructuras de control son la base del control del flujo de un programa.

Tipos comunes

1. Condicionales (if)
2. **Bucles** (`while`, `for`)

Preguntas

- ¿Cómo harías para pedir al usuario que ingrese información hasta que dé una respuesta válida?
- Si tuvieras que sumar 2 números, ¿cómo lo harías?
  - ¿Y si fuesen 10?
  - ¿Y si fueran 1000000?
- Si tuvieras que sumar los valores de una lista de 10000 elementos, ¿te gustaría tener que escribirlo manualmente?
- ¿Cómo harías para sumar un número de valores que cambia cada vez que se ejecuta el programa?

Ejemplos

- Pasar lista en clase.
  > Hay un cierto número de estudiantes en clase.
  >
  > Para cada estudiante de la clase tienen que decir su nombre.
- Una cesta tiene calcetines sin doblar. Se tienen que doblar todos los calcetines. Doblar calcetines de la cesta.
  > Mientras haya calcetines en la cesta.

Condición de salida

Una condición de salida en un bucle es la regla que indica **cuándo** el bucle debe **detenerse**.

- Cuando se hayan nombrado a todos los estudiantes de la clase.
- Cuando se haya hecho un número concreto de veces.
- Cuando **no** haya más calcetines.
- Cuando **no** queden más elementos que procesar.

> ⚠️ Garantiza que el bucle termine en algún momento.

Condición de salida

- **La condición de salida debe cambiar**: debe permitir entrar en la ejecución del bucle y debe permitir salir.
- **Repetir un número determinado de veces**.

Caso práctico

```
print("Hola, Juan.")
print("Hola, María.")
print("Hola, Pedro.")
```

- ¿Qué pasaría si en lugar de tres nombres, tuviéramos 100?
- ¿Cómo podríamos evitar escribir `print("Hola, XXX.")` tantas veces?

Bucles `for`

`for`

Un bucle `for` es una estructura de control que repite un bloque de instrucciones *una cantidad determinada de veces*.

> Para todos los elementos de un conjunto haz ...

Se usa para

- Repetir un bloque de instrucciones un número de veces.
- Recorrer elementos de una lista o colección.
- Un bucle `for` responde a: ¿Cuántas veces debo ejecutar este bloque?

Iteración

⚠️ Una **iteración** es una vuelta o repetición dentro del bucle.

En cada iteración, la variable del bucle toma un valor distinto y ejecuta el mismo bloque de instrucciones.

```
suma = 0

for numero in range(0, 5):
    suma = suma + numero

print(suma)
```

Estructura

```
for variable in secuencia:
    # Bloque de instrucciones que se repite
else:
    # Bloque que se ejecuta al finalizar el bucle
    # (solo si no hubo break)
```

Ejemplo

```
for i in range(3):
    print("Número:", i)
else:
    print("El bucle terminó correctamente")
```

```
Número: 0
Número: 1
Número: 2
El bucle terminó correctamente
```

Ejemplo

```
for i in range(5):
    if i == 3:
        print("Se ha alcanzado el número 3.")
    else:
        print("Número:", i)
print("Más instrucciones")
```

Ejemplo

```
for i in range(5):
    if i == 3:
        break
    print("Número:", i)
else:
    print("El bucle terminó correctamente")
print("Más instrucciones")
```

```
Número: 0
Número: 1
Número: 2
Más instrucciones
```

Función `range()`

Función que permite generar secuencias de números enteros. No crea una lista, si no que se crea cuando se necesitan.

- `range(fin)` : desde 0 a *fin-1*.
- `range(inicio, fin)` : desde *inicio* a *fin-1*
- `range(inicio, fin, paso)` : desde *inicio* a *fin-1*, con paso

```
for i in range(0, 5, 2):
    print(i)
```

```
0
2
4
```

Errores comunes con `for`

- Sintaxis.
- Indentación incorrecta.
- Mal uso de `range()`
- Modificar el contenido que se está iterando.
- Mal uso de la variable del bucle.

Cuándo es recomendable usar `for`

- Se conoce el número de veces.
- Recorrer todos los elementos de una colección: cadena, lista, diccionario...

Ejemplo:

1. Suma los números del 1 al 10.
2. Muestra todos los números pares hasta 20. Utiliza `for` y `range()`. Haz 2 versiones: una con `range()` y otra con `if` dentro del bucle.
3. Cuenta del 10 al 1.

Ejemplo: sumar números del 1 al 10

```
suma = 0
for i in range(1, 11):
    suma += i
print("La suma es:", suma)
```

Ejemplo: contar pares hasta 20

```
for i in range(0, 21, 2):
    print(i)
```

```
for i in range(0,21):
    if i % 2 == 0:
        print(i)
```

Ejemplo: contar del 10 al 1

```
for i in range(10, 0, -1):
    print(i)
```

Bucles `while`

`while`

Un bucle `while` es una estructura de control que repite un bloque de código *mientras una condición sea verdadera*.

> Si se cumple la condición haz las instrucciones y continúa mientras se cumpla esta condición.
> Mientras se cumpla la condición haz las intrucciones.

Estructura

```
while condición:
    # bloque de código que se repite
else:
    # bloque que se ejecuta si el bucle termina normalmente
```

Ejemplo

```
contador = 1
while contador <= 3:
    print("Número:", contador)
    contador += 1
else:
    print("El bucle terminó correctamente")
print("Más instrucciones")
```

```
Número: 1
Número: 2
Número: 3
El bucle terminó correctamente
Más instrucciones
```

Ejemplo

```
contador = 1
while contador <= 5:
    if contador == 3:
        break  # interrumpe el bucle
    print("Número:", contador)
    contador += 1
else:
    print("Bucle terminado correctamente")
print("Más instrucciones")
```

```
Número: 1
Número: 2
Más instrucciones
```

Ejemplos:

1. Haz un contador de 10 a 1. Usa `while` y `print()` para mostrar los números en orden descendente.
2. Suma los números del 1 al 10. Usa `while` y `print()` para mostrar la suma final.
3. Valida un valor de entrada. Pide al usuario que introduzca una nota entre 0 y 10. Si el valor introducido no es válido, vuelve a pedirlo hasta que sea correcto. Usa `while` y `input()` para pedir la nota.

Ejemplo: contador de 10 a 1.

```
contador = 10

while contador >= 1:
    print(contador)
    contador = contador - 1
```

```
10
9
8
7
6
5
4
3
2
1
```

Ejemplo: sumar números del 1 al 10

```
suma = 0
numero = 1

while numero <= 10:
    suma = suma + numero
    numero = numero + 1

print(suma)
```

> Cómo sería el ejemplo de contar del 10 al 1 con `while`?

Ejemplo: validar un valor de entrada

```
nota = -1

while nota < 0 or nota > 10:
    nota = float(input("Introduce una nota entre 0 y 10: "))

print("Nota válida:", nota)
```

Errores comunes con `for`

- Sintaxis.
- Condición mal formulada: no entra en el bucle.
- La condición de salida nunca es `False`: no sale del bucle, bucle infinito.

```
contador = 1

while contador <= 5:
    print(contador)
```

- Usos incorrectos de `break` o `continue`.

Cuándo es recomendable usar `while`

- No se conoce cuántas veces se repetirá el bucle.
- Cuando algo debe cambiar para detener el bucle.

Diferencias entre `for` y `while`

Característica	`for` loop	`while` loop
Uso principal	Iterar sobre secuencias/rangos	Repetir mientras condición
Control del ciclo	Por secuencia (`range`, lista...)	Por condición (`x < 5`, etc.)
Repeticiones	Conocidas/fijas	Desconocidas/variables
Ideal para	Recorrer listas, cadenas, etc.	Esperar eventos o condiciones
Bucle infinito	Poco probable	Más probable

Cuando usar `for` y `while`

Cuándo usar	`for`	`while`
Repeticiones conocidas	✅	❌
Recorrer secuencias	✅	❌
Condición dinámica	❌	✅
Bucle hasta evento	❌	✅
Riesgo de bucle infinito	Bajo	Alto

Ejemplo

```
for i in range(3):
    print("Hola")
```

vs

```
while respuesta != "salir":
    respuesta = input("Escribe 'salir' para terminar: ")
```

`break` y `continue`

`break` detiene el bucle en el punto en el que se encuentra (no evalúa nada más)

`continue` se usa dentro de un bucle para saltar a la siguiente iteración, omitiendo el resto del código que está después de `continue` en esa vuelta.

Comando	Ventajas	Desventajas
break	- Salir anticipadamente de un bucle. - Evita iteraciones innecesarias, mejorando eficiencia.	- **Dificulta la legibilidad**. - Puede interrumpir lógica que debía continuar.
continue	- Permite saltar iteraciones. - Útil para filtrar datos o saltar valores inválidos.	- **Dificulta la legibilidad**. - Puede confundir si hay varias condiciones `continue` dispersas. - Puede causar bucles infinitos si se omite actualizar variables de control.